

UNIT – I

Introduction to Python: Features of Python, History of Python, Need of Python Programming, Applications Basics of Python Programming Using the REPL(Shell), Running Python Scripts, Variables, Assignment, Keywords, Input-Output, Indentation.

Data types: Integers, Strings, Booleans.

History of Python:

- Python was created by **Guido Van Rossum**, who was a Dutch person from Netherlands in the Year 1991.
- Guido Van Rossum created it when he was working in National Research Institute of Mathematics and Computer Science (NRIM&CS) in Netherlands.
- He thought to create a scripting language **as a “Hobby”** in Christmas break in 1980. He studied all the languages like ABC (All Basic Code), C, C++, Modula-3, Smalltalk, Algol-68 and Unix Shell and collected best features from these languages.
- He started implementing it from 1989 and finished and released first working version of Python in 1991.
- He named it as **“Python”**, being a big fan of “Monty Python’s Flying Circus” comedy show broadcasted in BBC from 1969 to 1974.

➤ Python 1.0 (1994)

- ☐ Python 1.5(1997)
- ☐ Python 1.6 (2000)

➤ Python 2.0 (2000)

- ☐ Python 2.1 (2001)
- ☐ Python 2.2 (2001)
- ☐ Python 2.3 (2003)
- ☐ Python 2.4 (2004)
- ☐ Python 2.5 (2006)
- ☐ Python 2.6 (2008)
- ☐ Python 2.7 (2010)

➤ Python 3.0 (2008)

- ☐ Python 3.1(2009)
- ☐ Python 3.2(2011)
- ☐ Python 3.3 (2012)
- ☐ Python 3.4(2014)
- ☐ Python 3.5 (2015)
- ☐ Python 3.6 (2016)
- ☐ Python 3.7 (2018)
- ☐ Python 3.8 (2019)
- ☐ Python 3.9(2020)

Features of Python:

Simplicity: Python programming language is known for its simplicity. In comparison to other programming languages, Python is considered one of the easiest languages to learn. The syntax of this programming language is very simple and it can easily be understood.

Platform independent: Python is a platform-independent programming language. It means Python can run equally easily on a variety of platforms like Windows, Linux and Unix, and many more. So, there is no need to write a separate code for each Operating System, as the same code can run on multiple platforms.

Open source: The one useful feature of Python is that it is open-source. Using the Internet, anyone can easily download and use Python.

High-level language: This is one key feature of Python. This means no necessity for programmers to remember the system architecture and they also not required to manage memory as well.

Interpreted: Python is an interpreted programming language. It means interpreter executes the code line by line at a time, making debugging easier.

Library support: Large set of Python libraries are available to programmers that help them to easily write the code. A large set of libraries means that there is no need for programmers to write a code for every single thing.

Embedding properties: Python allows the code of other programming languages to be embedded, thus making it a more powerful programming language.

Object-Oriented concept: Python supports Object-Oriented concept. This feature provides a lot of flexibility for the developers to easily develop projects.

Need of Python Programming:

The following are the factors to use Python

- **Software quality** - Python code is designed to be readable, reusable and maintainable
- **Developer productivity** - Python boosts developer productivity many times beyond compiled or statically typed languages such as C, C++, and Java. Python code is typically one-third to one-fifth the size of equivalent C, C++ and Java Programs.
- **Program portability** - Most Python programs run unchanged on all major computer platforms. Porting Python code between Linux, Windows, and Mac will be easy.
- **Support libraries** - Python comes with a large collection of prebuilt and portable functionality, known as the standard library. This library supports an array of application-level programming tasks, from text pattern matching to network scripting.
- **Component integration** - Python scripts can easily communicate with other parts of an application (modules), using a variety of integration mechanisms. Such integrations allow Python to be used as a product customization and extension tool. Today, Python code can invoke C and C++ libraries.
- **Enjoyment** - It is easy to use. It gives programmer flexibility to write programs.

Who Uses Python Today?

1. Google makes extensive use of Python in its web search systems.
2. The popular YouTube video sharing service is largely written in Python.
3. The Dropbox storage service codes both its server and desktop client software primarily in Python.
4. The Raspberry Pi single-board computer promotes Python as its educational language.
5. The widespread BitTorrent peer-to-peer file sharing system began its life as a Python program.
6. Google's App Engine web development framework uses Python as an application language.
7. Maya, a powerful integrated 3D modeling and animation system, provides a Python scripting API.

8. Intel, Cisco, Hewlett-Packard, Seagate, Qualcomm, and IBM use Python for hardware testing.
9. NASA, Los Alamos, Fermilab, JPL, and others use Python for scientific programming tasks.

Applications:

Some of Python's most common applications today:

1. Systems Programming
2. GUIs
3. Internet Scripting
4. Component Integration
5. Database Programming
6. Rapid Prototyping
7. Numeric and Scientific Programming

Basics of Python Programming Using the REPL(Shell):

- An Interpreter is a program which translates your code into machine language and then executes it line by line.

Running Python in Three Ways:

- a) Running Python Interpreter.
- b) Running Python Scripts in Command Prompt
- c) Running Python Scripts in IDLE

Interactive Mode:

- In Interactive Mode, Python interpreter waits for you to enter command. When you type the command, Python interpreter goes ahead and executes the command, then it waits again for your next command.
 - Python interpreter in interactive mode is commonly known as Python Shell.
 - A **REPL** is an interactive mode to talk to your computer in Python. To make this work, the computer does four things:
 - Read** the user input (your Python commands).
 - Evaluate** your code (to work out what you mean).
 - Print** any results (you can see the computer's response).
 - Loop** back to step1 (to continue the conversation).
- The term "REPL" is an acronym for Read, Evaluate, Print and Loop

Script Mode(development mode)

This mode is used to write python script in a separate file under .py extension and to run the file

- Open the python shell and go to file→ new window
- Then type python code and save with .py extension
- The run the file using 'Run' button

Apart from **IDLE**, here are some of Python's most commonly used IDEs:

1. Eclipse and PyDev
2. Komodo
3. NetBeans IDE for Python
4. PythonWin
5. Wing, Visual Studio,
6. Jupiter
7. Pycharm and others

Variables and Assignment:

- A variable is a memory location where a programmer can store a value. Example: roll_no, amount, name etc.
- Value is either string, numeric etc.
Ex: roll_no=1234, name="abc"
- Variables must be assigned before being referenced.
- The value stored in a variable can be accessed or updated later.
- No declaration required
- The type (string, int, float etc.) of the variable is determined by Python
- The interpreter allocates memory on the basis of the data type of a variable.
- **Variable name rules:**
 - Must begin with a letter (a - z, A - Z) or underscore (_)
 - Other characters can be letters, numbers or _
 - Case Sensitive
 - Can be any (reasonable) length
 - There are some reserved words which you cannot use as a variable name because Python uses them for other things.

Assignment

- It is used to create a variable and make it reference to a value in the memory
- Variable name is written to the left of =, and value is written to the right of =
- Basic assignment statement in Python is

Syntax:

`<variable> = <expr>`

- Where the equal sign (=) is used to assign value (right side) to a variable name (left side).

```
>>> Item_name = "Computer" #A String
```

```
>>> Item_qty = 10 #An Integer
```

```
>>> Item_value = 1000.23 #A floating point
```

```
>>> print(Item_name)
```

```
Computer #output
```

```
>>> print(Item_qty)
```

```
10 #output
```

```
>>> print(Item_value)
```

```
1000.23 #output
```

- It is important to note that assignment statement read right to left only.

Example :

a = 12 is correct, but 12 = a does not make sense to Python, which creates a syntax error.

```
>>> a = 12
```

```
>>> 12 = a
```

SyntaxError: can't assign to literal

Multiple Assignment:

- The basic assignment statement works for a single variable and a single expression. You can also assign a single value to more than one variables simultaneously.

Syntax:

```
var1=var2=var3...varn= <expr>
```

Example :

```
x = y = z = 1
```

```
>>> x = y = z = 1
```

```
>>> print(x)
```

```
1      #output
```

```
>>> print(y)
```

```
1      #output
```

```
>>> print(z)
```

```
1      #output
```

- Here is an another assignment statement where the variables assign many values at the same time.

Syntax:

```
<var>, <var>, ..., <var> = <expr>, <expr>, ..., <expr>
```

Example :

```
x, y, z = 1, 2, "abcd"
```

- In the above example x, y and z simultaneously get the values 1, 2 and "abcd".

Swap variables:

Python swap values in a single line and this applies to all objects in python.

Syntax:

```
var1, var2 = var2, var1
```

```
>>> x = 10
```

```
>>> y = 20
```

```
>>> print(x)
```

```
10     #output
```

```
>>> print(y)
```

```
20     #output
```

```
>>> x, y = y, x
```

```
>>> print(x)
```

```
20     #output
```

```
>>> print(y)
```

```
10     #output
```

Local and global variables in python:

In Python, variables that are only referenced inside a function are implicitly global. If a variable is assigned a value anywhere within the function's body, it's assumed to be a local unless explicitly declared as global.

Program

```
var1 = "Python"
def func1():
    var1 = "PHP"
    print("In side func1() var1 = ",var1)
def func2():
    print("In side func2() var1 = ",var1)
func1()
func2()
```

Output

```
In side func1() var1 = PHP
In side func2() var1 = Python
```

- you can use global variable in functions by declaring it as global keyword

Program

```
def func1():
    global var1
    var1 = "PHP"
    print("In side func1() var1 = ",var1)
def func2():
    print("In side func2() var1 = ",var1)
func1()
func2()
```

Output

```
In side func1() var1 = PHP
In side func2() var1 = PHP
```

Keywords:

The following list shows the Python keywords. These are reserved words and you cannot use them as constant or variable or any other identifier names. All the Python keywords contain lowercase letters only.

- Type the following at interpreter to get the list of keywords

```
import keyword
print (keyword.kwlist)
```

And	except	not	assert
Finally	or	break	for
Pass	class	from	print
Continue	global	raise	def
If	return	del	import

Try	elif	in	while
else	is	True	False
Lambda	yield		

Input and Output functions:

- Python provides numerous built-in functions that are readily available to us at the python prompt.
- Some of the functions like input() and print() are widely used for standard input and output operations respectively. Let us see the output section first.

Python Input:

- Until now, our programs were static. The value of variables were defined or hard coded into the source code.
- To allow flexibility we might want to take the input from the user. In Python, we have the input() function to allow this.
- The syntax for input() is

```
input("prompt")
```
- Where prompt is the string we wish to display on the screen. It is optional.

```
>>> num = input('Enter a number: ')
Enter a number: 10
>>> num
'10'
```

Python Output Using print() function

- We use the print() function to output data to the standard output device (screen).

```
>>> print('This sentence is output to the screen')
This sentence is output to the screen      # Output
```
- ```
>>> a = 5
>>> print('The value of a is', a)
The value of a is 5 # Output
```
- In the second print() statement, we can notice that a space was added between the string and the value of variable a. This is by default, but we can change it.
- The actual syntax of the print() function is  

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```
- Here, objects is the value(s) to be printed.
- The sep separator is used between the values. It defaults into a space character. After all values are printed, end is printed. It defaults into a new line.
- The file is the object where the values are printed and its default value is sys.stdout(screen).

```
Ex: print(1,2,3,4)
Output: 1 2 3 4
print(1,2,3,4,sep='*')
Output: 1*2*3*4
print(1,2,3,4,sep='#',end='&')
Output: 1#2#3#4&
```

### Output formatting:

- Sometimes we would like to format our output to make it look attractive. This can be done by using the `str.format()` method. This method is visible to any string object.

```
>>> x = 5; y = 10
```

```
>>> print('The value of x is {} and y is {}'.format(x,y))
```

The value of x is 5 and y is 10 #output

- Here the curly braces `{}` are used as placeholders.
- We can specify the order in which it is printed by using numbers (tuple index)  

```
print('I love {0} and {1}'.format('bread','butter'))
```

I love bread and butter # Output

```
print('I love {1} and {0}'.format('bread','butter'))
```

I love butter and bread # Output
- We can even use keyword arguments to format the string.  

```
>>> print('Hello {name}, {greeting}'.format(greeting = 'Goodmorning', name = 'John'))
```

Output: Hello John, Goodmorning

- We can even format strings like the old `printf()` style used in C programming language. We use the `%` operator to accomplish this.

```
>>> x = 12.3456789
```

```
>>> print('The value of x is %3.2f' %x)
```

Output:

The value of x is 12.35

```
>>> print('The value of x is %3.4f' %x)
```

Output:

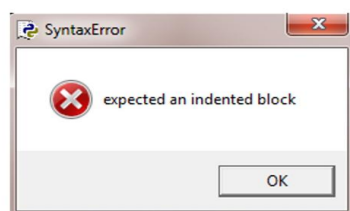
The value of x is 12.3457

### Indentation:

- Python uses whitespace (spaces and tabs) to define program blocks whereas other languages like C, C++ use braces `{}` to indicate blocks of codes for class, functions or flow control.
- The number of whitespaces (spaces and tabs) in the indentation is not fixed, but all statements within the block must be the indented same amount

### Program with Indentation Error

```
if True:
print("True")
else:
 print("False")
```

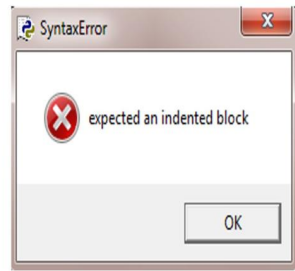


### Corrected Program

```
if True:
 print("True")
else:
 print("False")
```



```
if True:
 print("True")
else:
 print("False")
```



```
if True:
 print("True")
else:
 print("False")
```

```
if True:
 print("True1")
 print("True2")

else:
 print("False")
```



```
if True:
 print("True1")
 print("True2")
else:
 print("False")
```

## Data Types:

Python data types are:

1. Numbers
2. Booleans
3. Strings
4. List
5. Tuple
6. Set
7. Dictionary

### 1. Numbers:

- Integers, floating point numbers and complex numbers are fall under the Number category.
- These are defined as "int", "float", and "complex" class in Python. We can use the **type()** function to know type or class of a variable. Integers can be of any length, it is only limited by the memory variable. A floating point number is accurate up to 15 decimal points.

**Integers** or int are positive or negative numbers with no decimal point. Integers in Python 3 are of unlimited size.

e.g.

a= 100

b= -100

c= 1\*20

**#output**

print(a)      100

```
print(b) -100
print(c) 200
```

### Floating Values

It is a positive or negative real numbers with a decimal point.

```
a = 101.2
b = -101.4
c = 2.3*3
```

|          | #output            |
|----------|--------------------|
| print(a) | 101.2              |
| print(b) | -101.4             |
| print(c) | 6.8999999999999995 |

**Complex numbers** are combination of a real and imaginary part. Complex numbers are in the form of  $X+Yj$ , where X is a real part and Y is imaginary part.

```
a = complex(5) # convert 5 to a real part val and zero imaginary part
print(a)
b=complex(101,23) #convert 101 with real part and 23 as imaginary part
print(b)
Output :- (5+0j)
 (101+23j)
```

| Attribute       | Description                               |
|-----------------|-------------------------------------------|
| num.real        | Real component of complex number num      |
| num.imag        | Imaginary component of complex number num |
| num.conjugate() | Returns complex conjugate of num          |

### 2. Booleans:

- A Boolean variable can reference one of two values: **True or False**. Boolean variables are commonly used as flags, which indicate whether specific conditions exist.

```
b=False
print "The value of b is:",b
print "class of b is:",type(b)
b=True
print "The value of b is:",b
print "class of b is:",type(b)
```

### 3. Strings:

String is a sequence of characters. We can use single quotes or double quotes to represent the strings. Multiline strings are denoted with Triple Quotes, `'''` or `"""`.

- We can get the character at the specified position by its index. We can also get the substring from the index by specifying the range of the substring.
- The index always starts from zero. See the following example:  

```
s="Hello Python Programmer"
S[3]->returns 'l' character
```

```
>>> st1='aaaaaaaa bbbb cccc dddd eee ffff'
>>> st1
'aaaaaaaa bbbb cccc dddd eee ffff'
>>> st2="aaaaaaaa bbbb cccc dddd eee ffff"
>>> st2
'aaaaaaaa bbbb cccc dddd eee ffff'
>>> st3="""aaaaa bbbbbb
cccc dddd
eeee ffff"""
>>> st3
'aaaaa bbbbbb\ncccc dddd\neeee ffff'
>>> str = 'Hello World!'
```

|                     |                          |
|---------------------|--------------------------|
|                     | #output                  |
| print(str)          | Hello World!             |
| print(str[0])       | H                        |
| print(str[2:5])     | llo                      |
| print(str[2:])      | llo World!               |
| print(str * 2)      | Hello World!Hello World! |
| print(str + "TEST") | Hello World!TEST         |

**Updating Strings** You can "update" an existing string by (re)assigning a variable to another string. The new value can be related to its previous value or to a completely different string all together.

For example:

```
var1 = 'Hello World!'
print "Updated String :- ", var1[:6] + 'Python'
```

When the above code is executed, it produces the following result:

Updated String :- Hello Python

**String Special Operators** Assume string variable a holds 'Hello' and variable b holds 'Python', then:

a=Hello

b=Python

| Operator   | Description                                                                                                 | Example                                  |
|------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------|
| +          | Concatenation - Adds values on either side of the operator                                                  | a + b will give HelloPython              |
| *          | Repetition - Creates new strings, concatenating multiple copies of the same string                          | a*2 will give – HelloHello               |
| [ ]<br>[:] | Slice - Gives the character from the given index<br>Range Slice - Gives the characters from the given range | a[1] will give e<br>a[1:4] will give ell |